

F1 Top-10 Finish Prediction with Machine Learning

CX016-2.5-3-IML - Introduction to Machine Learning
Group Assignment

Module	CX016-2.5-3-IML - Introduction to Machine Learning
Deliverable	Report, notebook, dataset, scripts and submission ZIP
Main task	Predict whether each Formula 1 driver finishes in the top 10
Dataset	6,999 driver-race rows, 198 variables, seasons 2010-2026
Primary model	Histogram Gradient Boosting classifier

Generated from report/Report.md using scripts/build_report_pdf.py

Table of Contents

1. Abstract
2. Introduction, Aim and Objectives
3. Assignment Compliance Snapshot
4. Related Works
5. Methods
6. Dataset Preparation and Exploratory Data Analysis
7. Code Organization and Reproducibility
8. Model Implementation
9. Model Validation
10. Analysis and Recommendations
11. Conclusion
12. Acknowledgements
13. References
14. Appendix: Neural Network and Ranking Extensions

Abstract

This project investigates whether machine learning can predict if a Formula 1 driver will finish a Grand Prix in the top 10. This is a useful sporting analytics problem because top-10 classification is directly linked to points scoring, race strategy and team performance evaluation. A single model-ready dataset was built from public Formula 1 data covering 2010 to 2026, combining race results, qualifying, standings, circuit information, historical pit-stop indicators, weather, and selected FastF1-derived features. The dataset was cleaned, merged and transformed into 6,999 driver-race observations with 198 variables and no missing values. The main predictive task is binary classification with `top10_finish` as the target. Logistic regression, random forest, extra trees, histogram gradient boosting and a multilayer perceptron were compared using temporal validation. The best holdout model is histogram gradient boosting, with 2025 race `precision@10` of 0.779. A separate neural-network regressor was also trained as a ranking extension for predicted finishing order, but the primary assignment result remains the comparison and validation of predictive models on the tabular top-10 classification task.

Introduction, Aim and Objectives

Formula 1 results are influenced by many interacting factors: car performance, driver ability, qualifying position, reliability, circuit type, weather and race strategy. This makes the problem more challenging than a simple ranking by previous points. The goal of this project is to use historical and pre-race information to estimate whether each driver will finish inside the top 10.

The aim is to build, evaluate and compare machine learning models for Formula 1 top-10 finish prediction.

The objectives are:

- collect and consolidate a reasonably large, non-trivial Formula 1 dataset
- include both numeric and categorical features
- perform exploratory data analysis and preprocessing
- engineer predictive features that avoid direct post-race leakage
- train and compare at least two predictive machine learning models
- include model tuning and validation
- interpret the results and recommend the most suitable model
- provide reproducible code, generated figures and a final submission archive

The scope is supervised prediction at driver-race level. The main output is a probability and ranking of likely top-10 finishers. Exact race simulation, live telemetry modelling and lap-by-lap race strategy optimization are treated as future work rather than core assignment requirements.

Assignment Compliance Snapshot

The assignment question requires one challenging dataset, predictive models, preprocessing, EDA, tuning, validation, interpretation, a report and source code. The current project satisfies these requirements as follows.

Assignment requirement	Project evidence	Status
One dataset	data/final/f1_top10_model_dataset.csv is the single model-ready dataset used for modelling	Satisfied
Reasonable size	6,999 driver-race rows, 198 variables and seasons 2010-2026	Satisfied
Not perfectly clean	Raw API tables require identifier matching, missing-data handling, external weather joins and partial FastF1 coverage	Satisfied
Numeric and categorical variables	Final dataset includes 174 numeric/bool columns and 24 categorical/text columns	Satisfied
More than 12 variables	The final dataset has 198 columns	Satisfied
At least two predictive models	Logistic regression, random forest, extra trees, histogram gradient boosting and MLP were compared	Exceeded

Assignment requirement	Project evidence	Status
Dataset preparation and EDA	Figures in outputs/figures/discuss target balance, season coverage, grid/finish relation and missing values	Satisfied
Optimization/tuning	MLP tuning, fixed ensemble hyperparameters and temporal validation are documented	Satisfied
Model validation	2025 holdout plus expanding-window rolling backtest	Satisfied
Report and compressed submission	report/Report.docx, report/Report.pdf and submission/IML_Assignment_GroupX.zip	Satisfied
Source code in notebook format	notebooks/ML_Project_Code.ipynb provides the assignment-facing notebook	Satisfied

Related Works

Previous Formula 1 analytics work often focuses on lap-time prediction, race outcome prediction, tire degradation or separating driver and constructor effects. These studies informed the feature choices in this project: qualifying position and team strength are important baseline predictors, while circuit, weather, tire and historical form features can improve the model.

Source	Focus	Relevance to this project
Breiman (2001)	Random forests	Motivates using tree ensembles for mixed tabular data
Friedman (2001)	Gradient boosting machines	Supports boosting as a strong tabular baseline
Pedregosa et al. (2011)	scikit-learn	Main software library used for modelling pipelines
Nigro (2020)	F1 race predictor	Shows practical race prediction using historical F1 data
Zhao (2024)	Deep neural network lap-time forecasting	Shows neural networks can model F1 performance signals
Jafri (2024)	F1 race outcome prediction	Uses historical race data for race prediction
Noe and Patel (2024)	Telemetry-based lap prediction	Motivates lap and performance features

Source	Focus	Relevance to this project
Cappello and Hoegh (2026)	Tire degradation modelling	Supports the value of tire and strategy variables
race-outcome decomposition work (2025)	Driver vs constructor effects	Motivates separating driver and constructor strength
Jolpica F1 API	Ergast-compatible data source	Provides race, qualifying, standings and pit-stop data
FastF1	Timing, lap and race-control data	Optional enrichment for recent seasons
Open-Meteo Archive	Historical weather	Adds race-day weather context

Compared with much of the related work, this project does not try to predict the race winner only. It predicts top-10 classification for every driver-race row, which creates a larger supervised dataset and aligns with the points-scoring structure of modern F1.

Methods

The final dataset is `data/final/f1_top10_model_dataset.csv`. Each row represents one driver in one race. The label is `top10_finish`, equal to 1 when the final classified result is tenth place or better, and 0 otherwise.

Item	Value
Driver-race observations	6,999
Seasons covered	2010-2026
Final variables	198
Numeric/bool variables	174
Categorical/text variables	24
Missing values after preprocessing	0
Top-10 rows	3,330
Non-top-10 rows	3,669

The main software packages are:

- pandas and numpy for data manipulation
- scikit-learn for preprocessing, pipelines, model training and metrics
- matplotlib and seaborn for EDA figures
- joblib for model serialization
- Plotly for optional neural-network visualization
- python-docx and ReportLab for generated report exports

The main models compared are:

Model	Role in project	Reason for inclusion
Logistic regression	Baseline classifier	Provides a simple interpretable linear benchmark
Random forest	Tree ensemble classifier	Strong tabular baseline and stable rolling-backtest model
Extra trees	Randomized tree ensemble	Tests whether more randomized tree splits improve robustness
Histogram gradient boosting	Champion classifier	Handles non-linear tabular interactions and achieved the best 2025 top-10 score
MLP classifier	Neural-network extension	Tests whether a dense non-linear neural architecture improves classification

The primary evaluation metrics are accuracy, precision, recall, F1, ROC-AUC and race precision@10. Race precision@10 is important because the practical output is a race-level top-10 list rather than only independent row-level labels.

Dataset Preparation and Exploratory Data Analysis

The assignment requires one reasonably large dataset with categorical and numeric data. Although several public sources were used, they are merged into one final modelling dataset. The raw tables are retained for reproducibility, while the final CSV is the single dataset used for modelling.

The dataset is not perfectly clean in its original form. Race APIs use different identifiers, some historical pit-stop data is unavailable before 2011, FastF1 timing coverage is partial, and weather data is an external historical approximation rather than exact FIA sensor data. The preprocessing pipeline handles these issues by using availability flags, deterministic fallbacks, imputation and careful feature merging.

Preparation step	Purpose
Merge raw tables	Combine race results, qualifying, standings, circuits, weather and optional FastF1 features
Create target	Build top10_finish from final classified race position
Remove leakage	Exclude post-race columns such as points, final position, fastest lap and same-race race-control counts during training
Impute numeric values	Fill missing numeric features with median values
Impute categorical values	Fill missing categorical features with the most frequent category

Preparation step	Purpose
One-hot encode categories	Convert driver, constructor, circuit and weather categories into model-ready features
Scale numeric values	Standardize numeric variables for logistic regression and neural-network models

Key EDA observations:

- The target is reasonably balanced for a binary classification task: about 48% top-10 rows and 52% non-top-10 rows.
- Rows per season vary because the number of races and drivers changes over time.
- Qualifying/grid position is strongly related to finishing position, but it is not enough alone because reliability, circuit, weather and team performance can change the outcome.
- Categorical fields such as driver, constructor, circuit and weather condition require encoding before model training.

Every figure included in the generated report is discussed in the relevant body section. The assignment-facing PNG figures summarize the full pipeline, holdout algorithm results, validation metric table and expanding-window stability.

Code Organization and Reproducibility

The codebase is organized so that data import, feature engineering, model definition, training and reporting can be inspected separately.

Path	Purpose
scripts/generate_raw_data.py	Imports Jolpica race, qualifying, driver, constructor, circuit and pit-stop data
scripts/generate_historical_weather.py	Adds Open-Meteo race-day weather features
scripts/generate_fastf1_features.py	Adds optional FastF1 timing, weather, tyre and lap-derived features
scripts/generate_fastf1_race_control.py	Imports race-control messages and builds disruption indicators
scripts/generate_final_dataset.py	Merges all raw and derived tables into the final modelling dataset
scripts/algorithms/classification.py	Contains top-10 classifier definitions and hyperparameters
scripts/algorithms/regression.py	Contains finish-position ranking model definitions
scripts/algorithms/neural_network.py	Contains dedicated MLP neural-network configurations

Path	Purpose
scripts/train_model.py	Trains the selected top-10 classifier and saves metrics/figures
scripts/evaluate_models.py	Compares all classifiers and runs rolling backtests
scripts/tune_neural_network.py	Tunes the dedicated MLP classifier
scripts/train_position_model.py	Trains the finish-position ranking models
scripts/build_report_docx.py	Generates the Word report
scripts/build_report_pdf.py	Generates the PDF report
scripts/validate_project.py	Checks that required files, outputs and metrics exist

The main reproducibility commands are:

```
python scripts/generate_final_dataset.py
python scripts/evaluate_models.py
python scripts/train_model.py --model hist_gradient_boosting
python scripts/tune_neural_network.py --force
python scripts/train_position_model.py
python scripts/make_charts.py
python scripts/build_report_docx.py
python scripts/build_report_pdf.py
python scripts/build_submission.py
python scripts/validate_project.py
```

Code comments are kept concise and focused. The main documentation is provided through README.md, docs/ALGORITHMS.md, docs/ASSIGNMENT_ALIGNMENT.md, the notebook and this report, so the submitted code remains readable without turning the report into a large code listing.

Model Implementation

For the main classification task, the selected champion model is histogram gradient boosting. This model was selected because it achieved the best 2025 holdout race precision@10 and handles non-linear interactions in tabular data well. Random forest remains the most stable model in rolling backtest, so it is kept as an important comparison baseline.

Model	Important parameters	Notes
Logistic regression	max_iter=1500, class_weight=balanced	Baseline model with scaled numeric features
Random forest	n_estimators=500, min_samples_leaf=3	Stable ensemble baseline
Extra trees	n_estimators=500, min_samples_leaf=3	More randomized tree ensemble
Histogram gradient boosting	max_iter=300, learning_rate=0.05, max_leaf_nodes=31, l2_regularization=0.05	Main champion classifier

Model	Important parameters	Notes
MLP classifier	hidden layers tested from (64, 32) to (128, 64, 32)	Additional neural-network experiment

As an additional experiment beyond the two required predictive models, a Multi-Layer Perceptron neural network was implemented using scikit-learn's `MLPClassifier`. The purpose was to test whether a non-linear neural architecture could capture more complex relationships in the dataset compared with traditional machine learning models. Several neural configurations were tested by varying the number of hidden layers, number of neurons, activation function, learning rate and L2 regularization parameter. Early stopping was also used to reduce overfitting by monitoring validation performance during training.

The MLP model used dense numerical input after preprocessing. Numerical variables were scaled, categorical variables were encoded, and the final transformed feature matrix was passed to the neural network. Since neural networks are sensitive to feature scale, standardization was applied to numerical features. The best current MLP classifier uses hidden layers (128, 64), ReLU activation, `alpha=0.002`, `learning_rate_init=0.0005`, `batch_size=64` and early stopping. After preprocessing, the model receives about 750 transformed features and has approximately 104,449 trainable parameters.

Model Validation

A temporal validation strategy is used instead of a random split. This is more realistic because a model should learn from past seasons and predict future races. The main holdout test season is 2025. An expanding-window rolling backtest is also used to compare stability across seasons.

Current holdout results on the 2025 season:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Race precision@10
Histogram gradient boosting	0.785	0.789	0.779	0.784	0.831	0.779
Random forest	0.779	0.762	0.813	0.786	0.844	0.775
Neural network MLP	0.766	0.756	0.788	0.771	0.829	0.771
Extra trees	0.752	0.735	0.788	0.761	0.833	0.750
Logistic regression	0.743	0.850	0.592	0.698	0.811	0.746

Training-time measurements on the current machine show that a single holdout run is lightweight. The heavier steps are the rolling backtest and full neural-network tuning because they train several models repeatedly.

Algorithm	Single holdout run time
Logistic regression	0.35 s
Random forest	1.66 s
Extra trees	2.05 s
Neural network MLP	2.46 s
Histogram gradient boosting	9.00 s

Rolling backtest averages show that random forest is the most stable model by race precision@10, with an average of 0.767. Histogram gradient boosting is still selected as the holdout champion because it has the strongest latest-season top-10 performance.

Analysis and Recommendations

The results are broadly in line with expectations. Tree-based models perform strongly because the dataset is tabular, mixed-type and feature-engineered. Logistic regression is useful as a simple baseline but is less able to capture non-linear interactions such as driver-team-circuit effects. The neural-network classifier is competitive but does not clearly beat tree ensembles on this dataset. This supports the decision to present the neural network as an extension rather than the primary assignment result.

Adding the 2010 season increased the dataset from 6,543 to 6,999 rows. The holdout accuracy and F1 improved for the champion classifier, while race precision@10 stayed at 0.779. This suggests that extra historical data helps general classification stability, but very old F1 seasons are not always directly representative of modern seasons.

The separate finish-position ranking model is useful for producing ordered race predictions. Its best current version is a neural-network MLP regressor with race precision@10 of 0.779 and mean race Spearman correlation of 0.660. This is helpful for interpretation, but it should remain secondary because the assignment asks mainly for predictive model comparison.

Recommended final model:

- main assignment model: histogram gradient boosting classifier
- supporting baseline: random forest classifier
- optional extension: tuned MLP classifier and finish-position regressor

Generated report deliverables:

- report/Report.md: editable report source
- report/Report.docx: Word report version
- report/Report.pdf: PDF report version generated from the same source
- outputs/figures/*.png: labelled EDA, algorithm and validation figures used as report evidence
- submission/IML_Assignment_GroupX.zip: compressed submission archive

Recommended future improvements:

- add calibration plots for probability quality
- add permutation importance for model interpretability
- fill remaining FastF1 gaps after rate limits reset
- test whether older seasons should be down-weighted because modern F1 regulations differ
- improve the report with more domain-specific literature if more time is available

Conclusion

This project satisfies the core assignment requirement by building a machine learning solution on one reasonably large, mixed-type and non-trivial dataset. The work includes data collection, cleaning, feature engineering, EDA, preprocessing, multiple predictive models, tuning, validation and comparison. The strongest model for the main top-10 classification task is histogram gradient boosting, with 2025 holdout race precision@10 of 0.779. The project also shows that neural networks are interesting but not automatically superior for this tabular problem. The main weakness is that some racing signals, especially full telemetry and older pit-stop details, are incomplete in public data. Overall, the project is a solid V0 for the assignment, with neural-network and 3D visualization work best positioned as optional extensions.

Acknowledgements

This project uses publicly available motorsport and weather data from Jolpica, FastF1 and Open-Meteo. The implementation relies on open-source Python libraries including pandas, numpy, scikit-learn, matplotlib, seaborn, Plotly, joblib, python-docx and ReportLab. The assignment structure follows the CX016-2.5-3-IML group assignment brief provided in the local docs/ folder.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5-32.
- Cappello, C., & Hoegh, A. (2026). A state-space approach to modeling tire degradation in Formula 1 racing. *Journal of Quantitative Analysis in Sports*.
- FastF1. (2026). FastF1 documentation. <https://docs.fastf1.dev/>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189-1232.
- Jafri, A. (2024). Predicting Formula 1 race outcomes: A machine learning approach. <https://aliabdullahjafri.com/>
- Jolpica. (2026). Jolpica F1 API. <https://github.com/jolpica/jolpica-f1>
- Nigro, V. (2020). Formula 1 race predictor: A machine learning approach. *Towards Data Science*.

Noe, C., & Patel, N. (2024). Utilizing telemetry data for machine learning-driven lap prediction. University of Rochester.

Open-Meteo. (2026). Historical Weather API. <https://open-meteo.com/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Zhao, Y. (2024). Deep neural network-based lap time forecasting of Formula 1 racing. *Applied and Computational Engineering*.

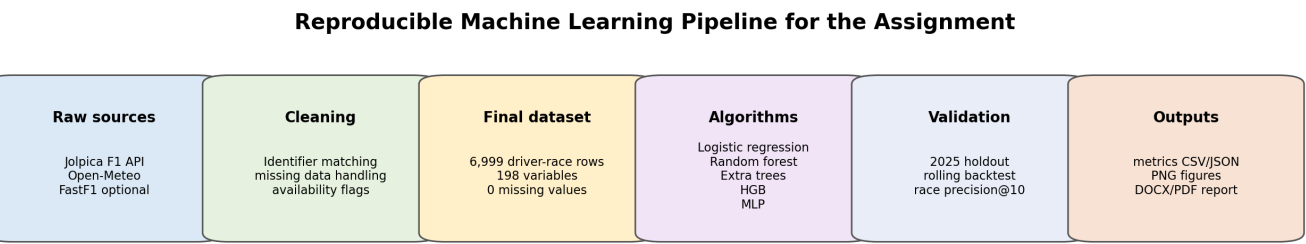
Appendix: Neural Network and Ranking Extensions

The neural-network work is included as an extension rather than the main assignment claim. The binary MLP classifier is implemented in `scripts/algorithms/neural_network.py` and trained through `scripts/tune_neural_network.py`. The 3D neural visualization is generated by `scripts/visualize_neural_network_3d.py` and exported to `outputs/figures/neural_network_embedding_3d.html`.

The finish-position ranking extension is implemented in `scripts/train_position_model.py` and `scripts/algorithms/regression.py`. It predicts an ordered finishing rank for each driver and is used in upcoming-race prediction exports. This extension is useful for practical race prediction but should be treated as secondary to the assignment's main supervised classification task.

Figures

Figure 1. End-to-end project pipeline from raw data sources to validated assignment outputs.



This figure links the required assignment steps: data collection, preprocessing, model implementation, validation and final report evidence.

Figure 2. Target distribution for the binary top-10 classification task.

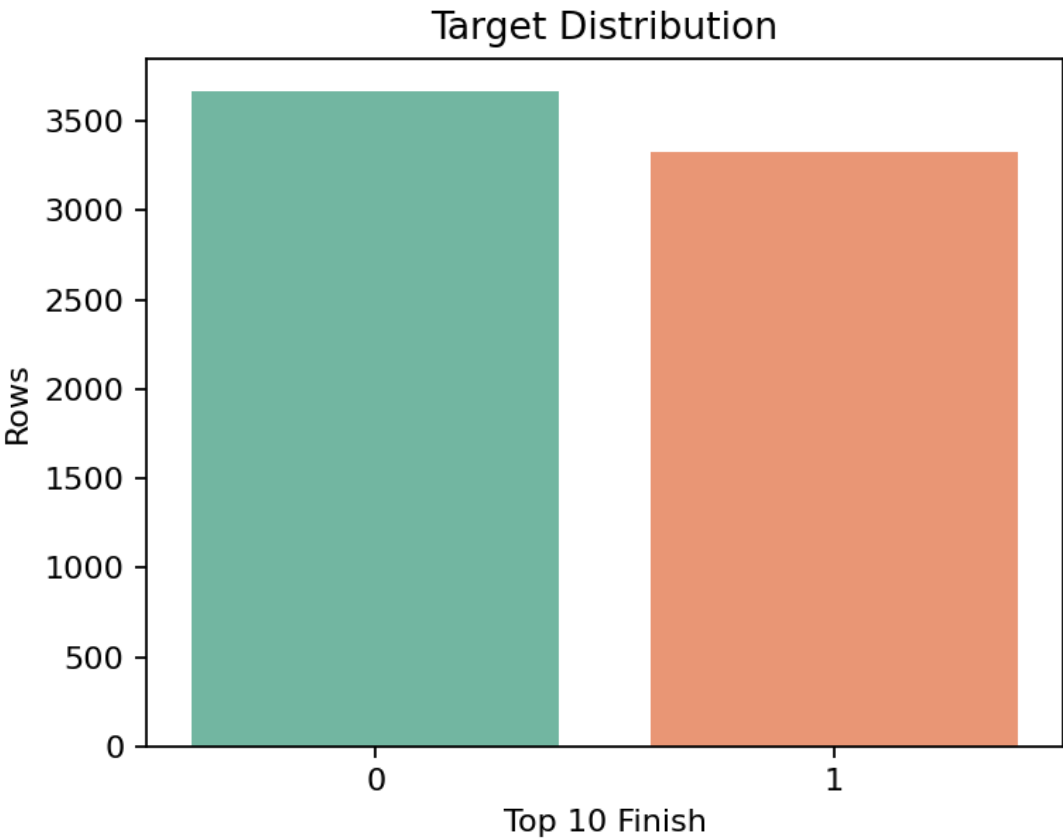


Figure 3. Number of driver-race rows available per season.

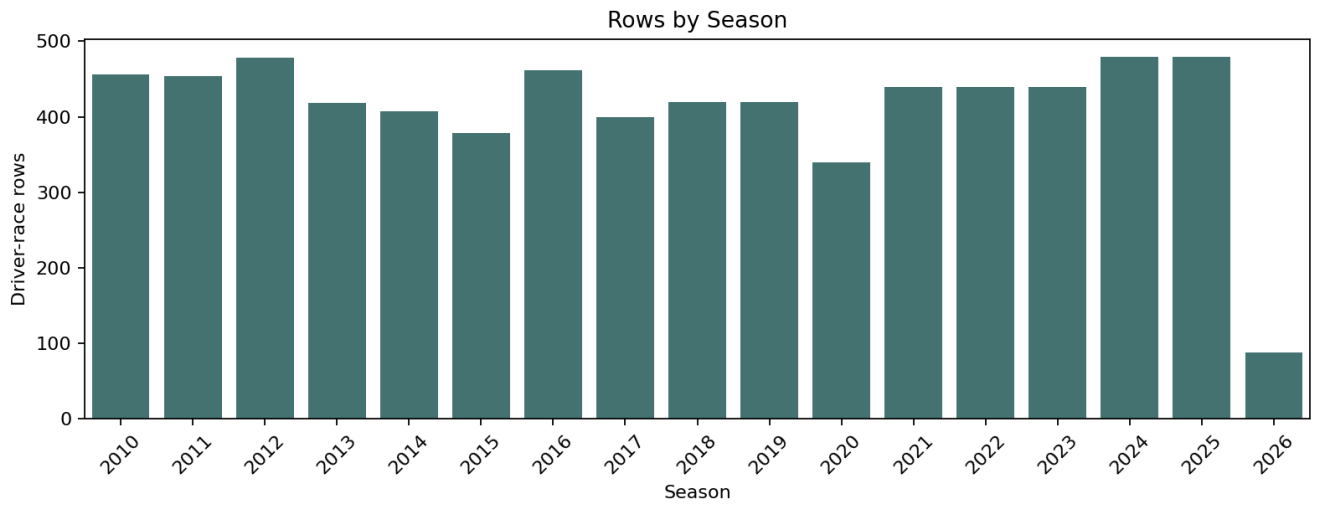


Figure 4. Top-10 target rate by season, used to check class stability over time.

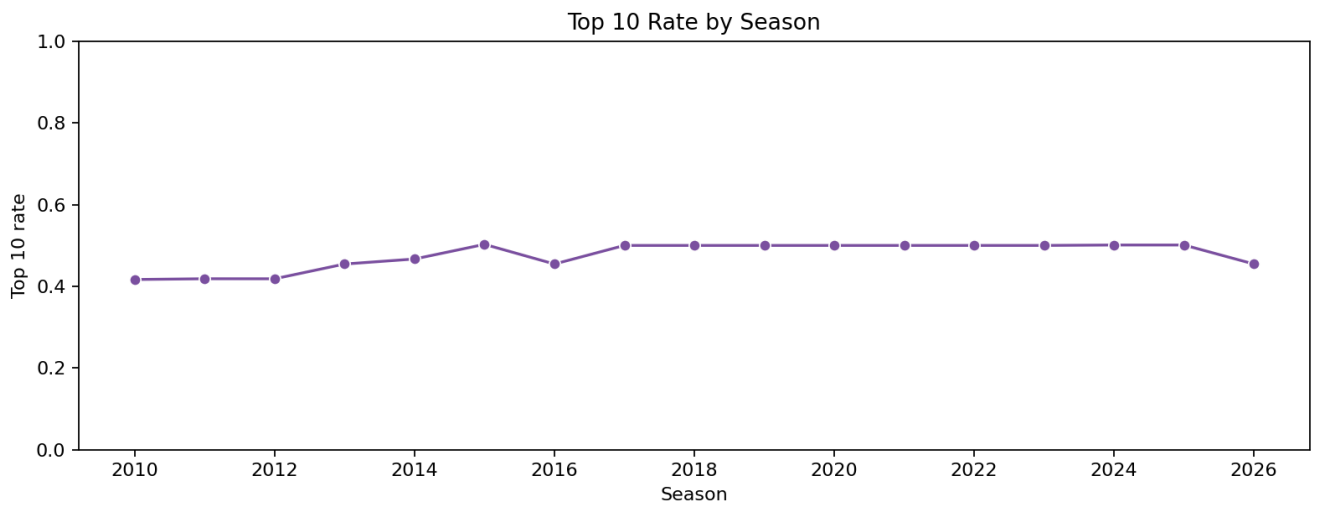


Figure 5. Relationship between starting grid and final position.

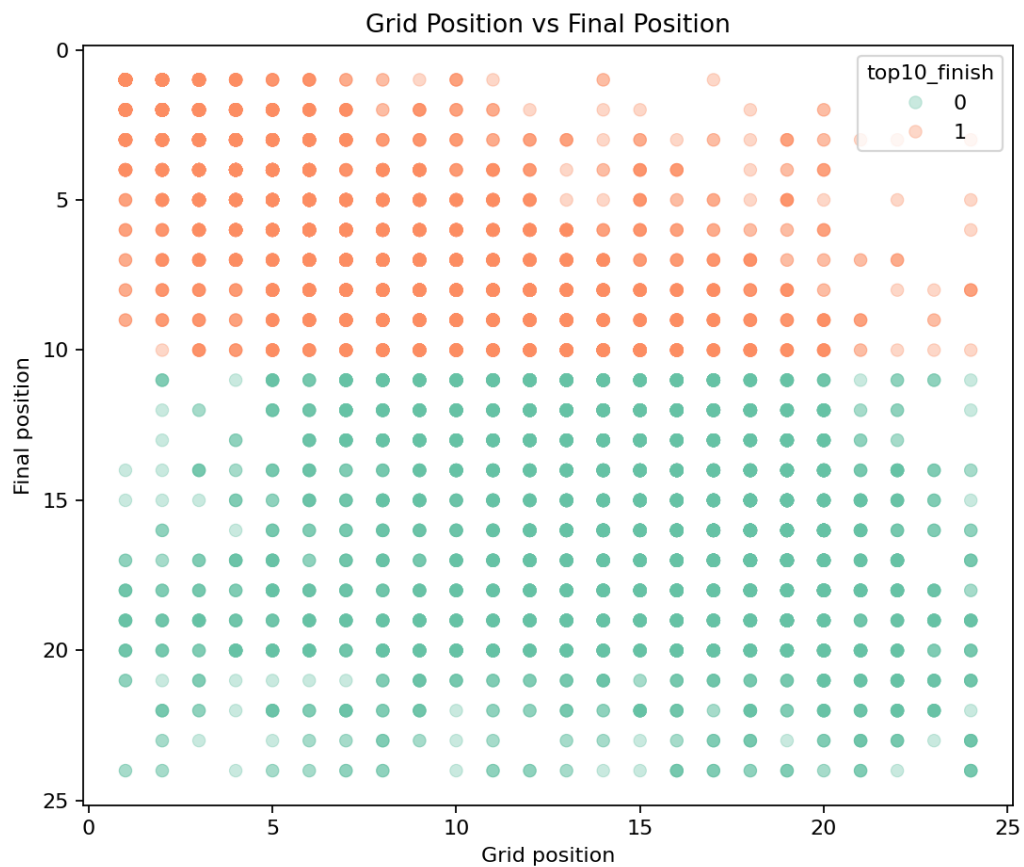


Figure 6. Holdout comparison of the main classification algorithms on the 2025 season.

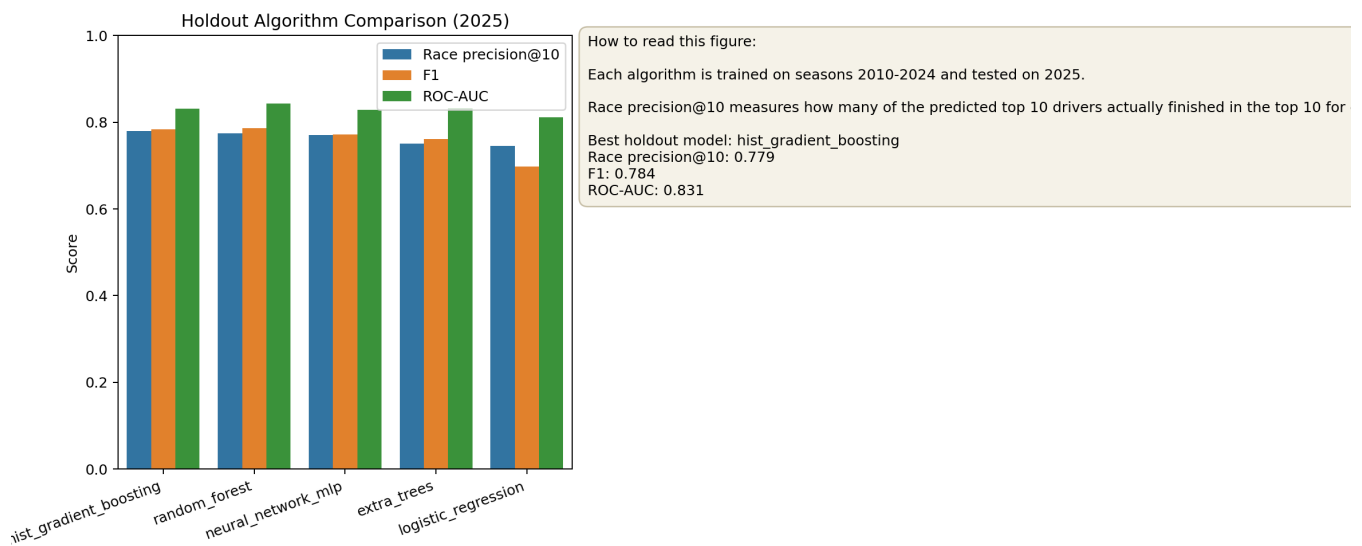


Figure 7. Validation metric table for the compared classification algorithms.

Model Validation Metrics on 2025 Holdout

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Race P@10
hist_gradient_boosting	0.785	0.789	0.779	0.784	0.831	0.779
random_forest	0.779	0.762	0.812	0.786	0.844	0.775
neural_network_mlp	0.766	0.756	0.787	0.771	0.829	0.771
extra_trees	0.752	0.735	0.787	0.761	0.833	0.750
logistic_regression	0.743	0.850	0.592	0.698	0.811	0.746

The table is sorted by race precision@10, the metric most aligned with predicting a race-level top 10.

Figure 8. Compact model comparison across F1, ROC-AUC and race precision@10.

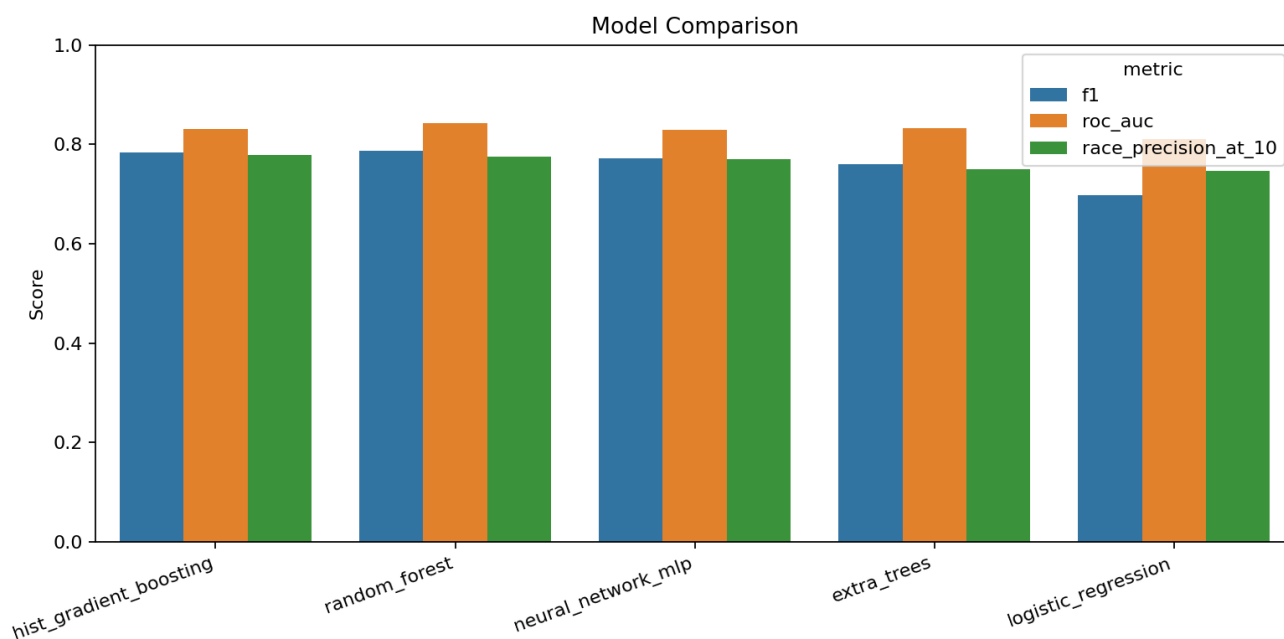


Figure 9. Expanding-window validation showing model stability across seasons.

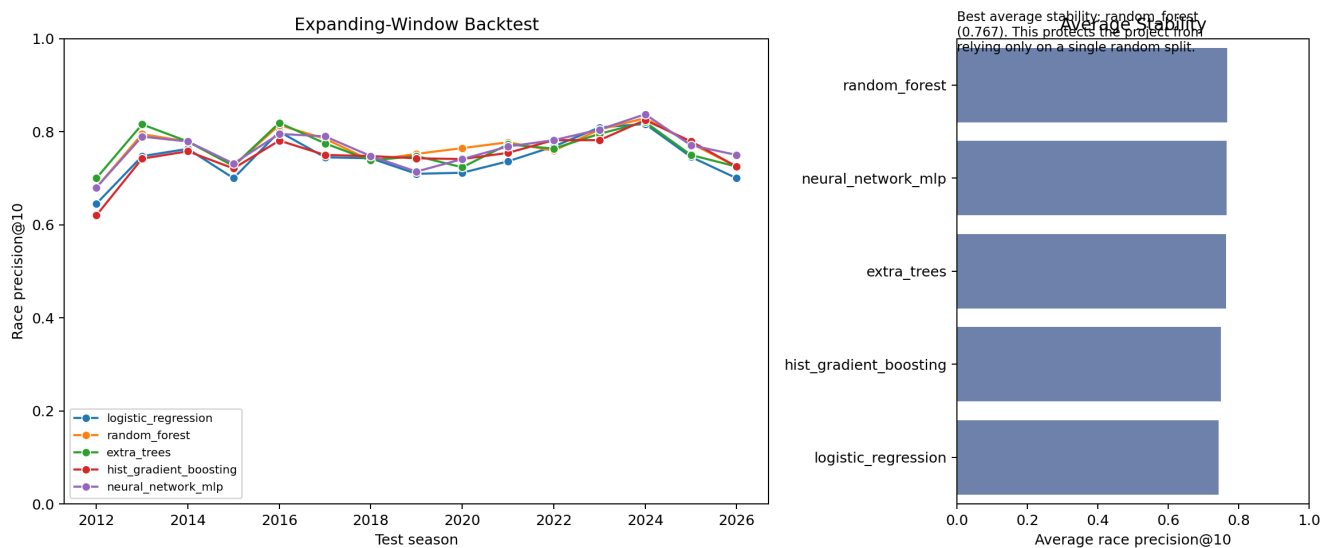


Figure 10. Rolling backtest race precision@10 by model and test season.

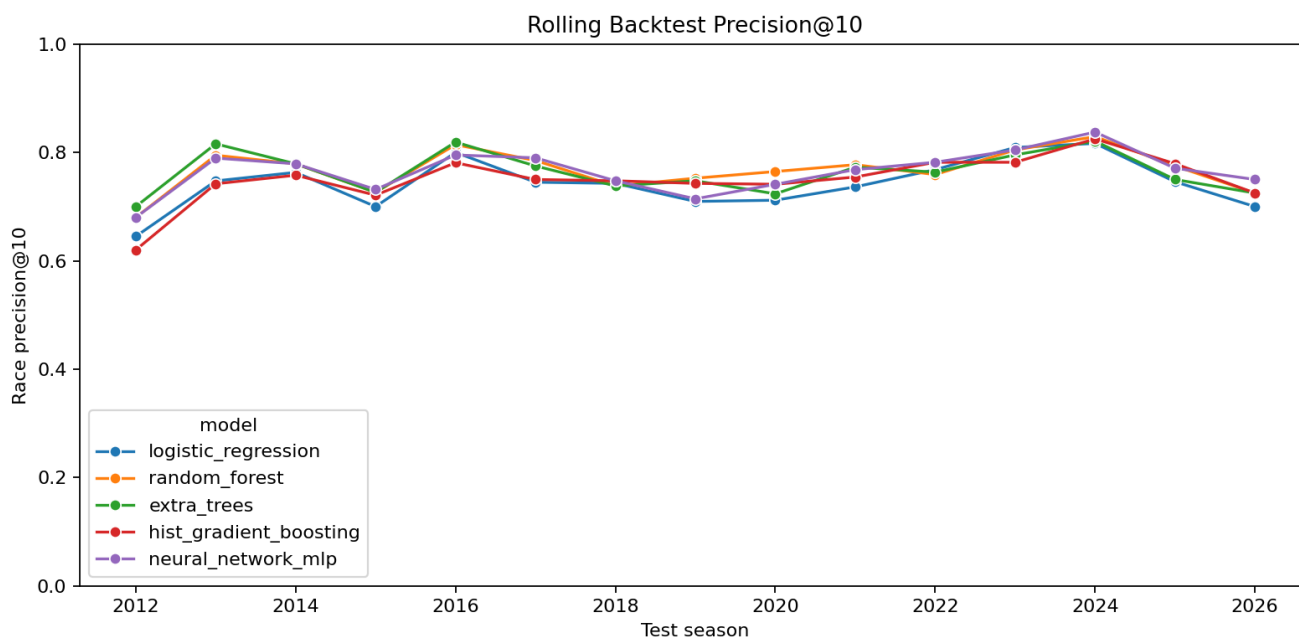


Figure 11. Most influential features for the selected top-10 classifier.

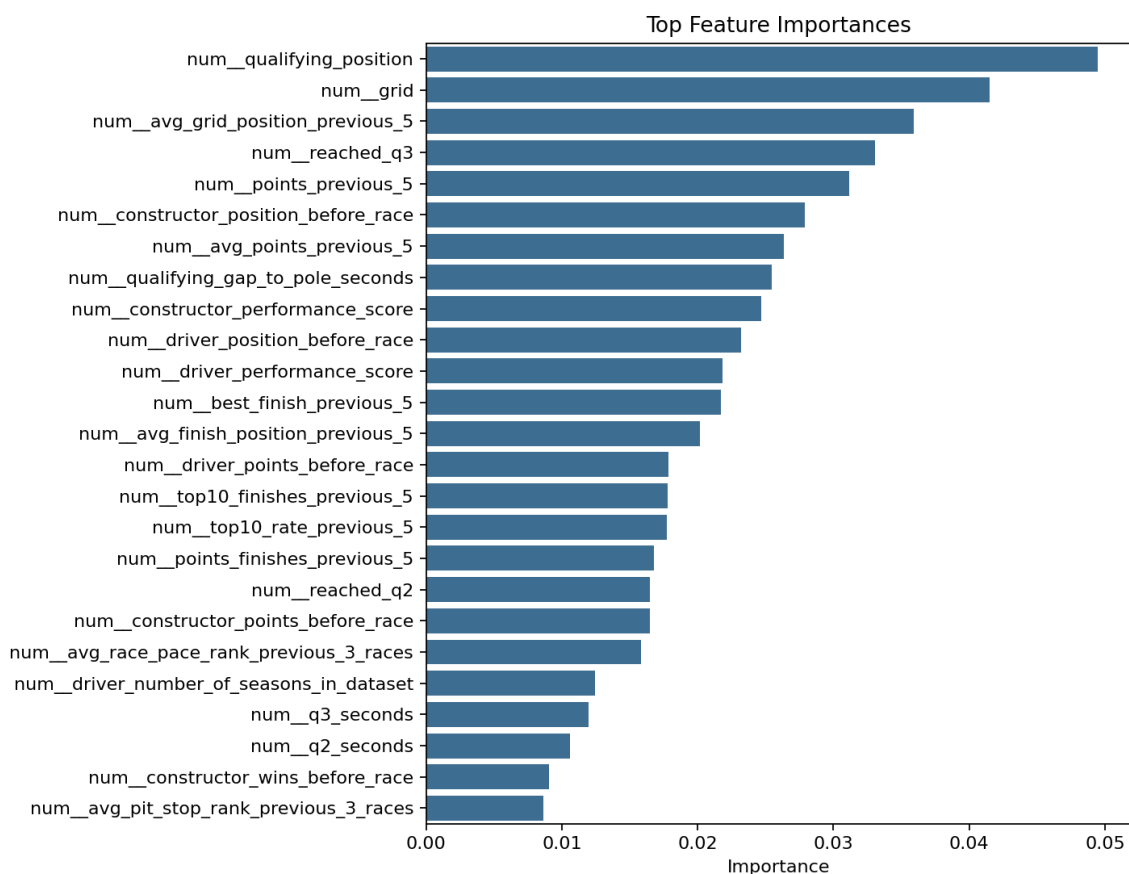


Figure 12. Confusion matrix for the selected holdout classifier.

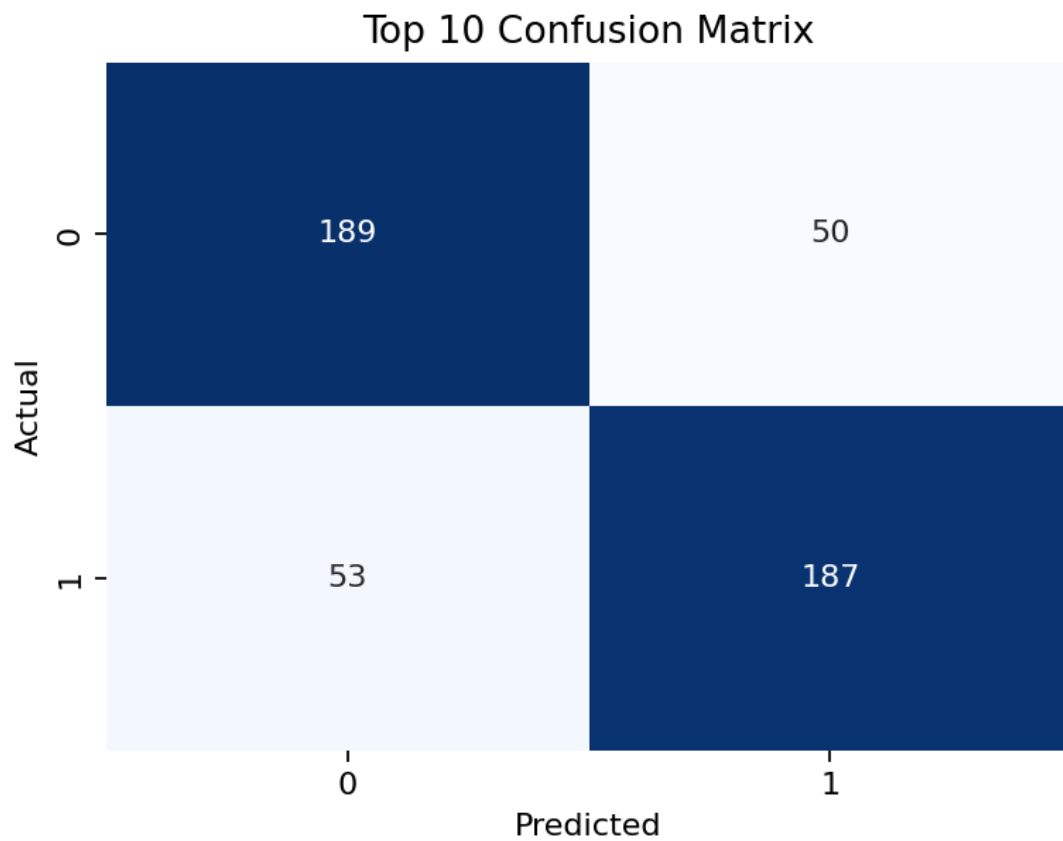


Figure 13. Neural-network classifier tuning results, included as an extension.

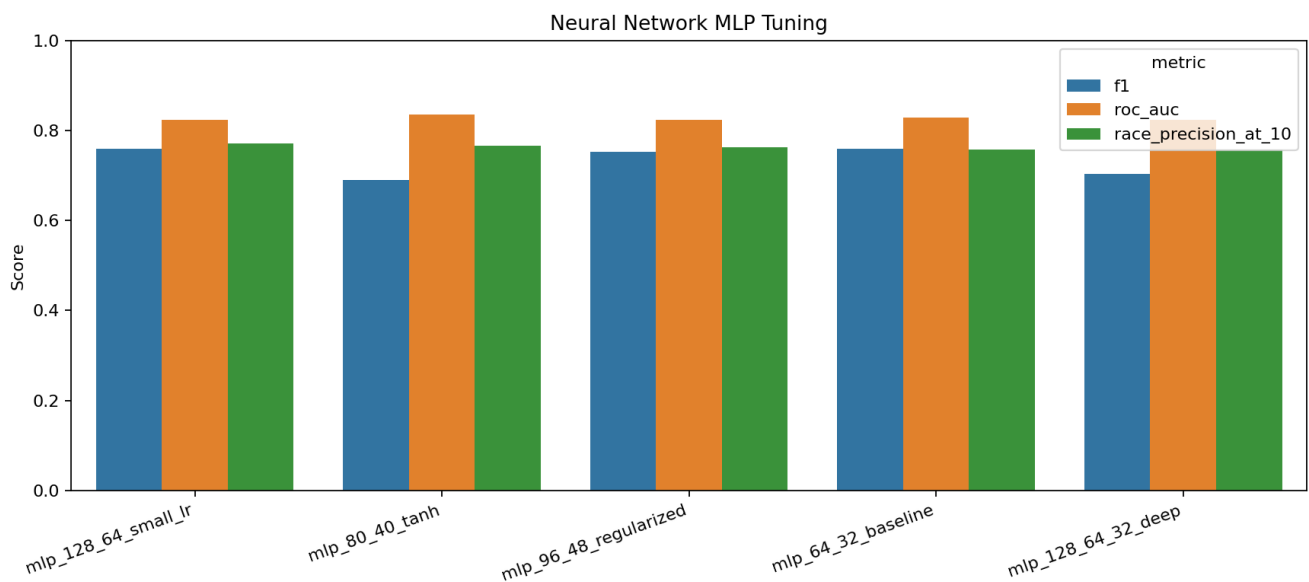


Figure 14. Finish-position ranking model comparison, included as an extension.

